Fast Failover & Resilience for Campus Networks using SDN

Complete Project Document & User Manual

Table of Contents

1. Project overview
2. System specifications
3. Architecture & topology design
4. Software stack & environment
5. Repository layout
6. How the system works
7. Resilience: the two failover mechanisms
8. Measured results
9. Installation & first-time setup
10. Operating the system — Mininet side
11. Operating the system — dashboard side
12. REST API reference
13. Full demonstration walkthrough
14. Validation & testing
15. Troubleshooting
16. Known limitations
17. Glossary

---

1. Project overview

This project builds a **realistic campus network** in software and makes it **self-healing**: when a link or switch fails, traffic automatically moves to a redundant path with minimal disruption, and the whole event is **detected, measured, and visualised** in real time.

It is built entirely on open-source SDN tooling:

- **Mininet** — emulates the switches, hosts, and links.
- **Open vSwitch (OVS)** — the software switches, speaking **OpenFlow 1.3**.
- **Ryu** — the SDN controller (the "brain") that programs the switches.
- **Python** — controller, topology model, experiments.
- **React + TypeScript** — the NOC (Network Operations Centre) dashboard.

The system demonstrates two complementary resilience mechanisms and **measures** each one:

Mechanism	Handles	Typical result
OpenFlow fast-failover groups (in the switch)	access-uplink, dist→core, core→dist link failures	~1 packet lost, ~20 ms
Controller rerouting (in the Ryu controller)	last-hop link failure, whole-switch failure	~2 packets lost, ~40 ms

Both recover in well under 100 ms, and every failure can be triggered — for real — from the Mininet CLI, a REST API call, or a click in the dashboard.

Why SDN? A traditional network reacts to failures hop-by-hop with slow distributed protocols. Here the controller has a **network-wide view**, computes **node-disjoint primary/backup paths**, pre-installs **fast-failover groups** for instant local protection, and **reroutes globally** for failures that local protection can't cover — all observable and measurable.

---

2. System specifications

2.1 Scale (full topology)

Layer	Count	Composition
Core switches	**4**	full mesh (6 core–core links)
Distribution switches	**12**	6 pods × 2 (plane A + plane B)
Access switches	**36**	6 pods × 6
Total switches	**52**	
Hosts	**72**	36 access × 2 hosts
Links	**174**	102 inter-switch + 72 host
Spanning-tree edges (for broadcast)	51	switches – 1
Discovered directed links (LLDP)	204	102 × 2 directions

2.2 Smoke topology (small, for safe demos)

`4 core + 4 distribution (2 pods × 2) + 4 access (2 pods × 2) = 12 switches`, `4 hosts`, `26 links`. Structurally identical (two planes, mesh core) — same code, smaller parameters.

2.3 Deterministic addressing

Everything is generated deterministically from one model, so IDs never drift.

Switch names & DPIDs

Type	Name	DPID formula	Example
Core	`c<i>`	`100 + i`	`c1` → 101
Distribution	`d<pod><a|b>`	`200 + pod*10 + (1|2)`	`d1a` → 211, `d6b` → 262
Access	`a<pod>_<idx>`	`1000 + pod*100 + idx`	`a1_1` → 1101, `a6_6` → 1606

Host addressing

Field	Formula	Example (`h6_6_1`)
Name	`h<pod>_<access>_<n>`	`h6_6_1`
IP	`10.<pod>.<access>.<n>`	`10.6.6.1`

Field	Formula	Example (`h6_6_1`)
MAC	`00:00:00:<pod>:<access>:<n>` (hex)	`00:00:00:06:06:01`

Deterministic port convention (contractual — the controller relies on it):

Switch	Ports
Access	`1` → plane-A dist (FF primary), `2` → plane-B dist (FF backup), `10+` → hosts
Distribution	`1`,`2` → core uplinks (FF primary/backup), `10+` → access downlinks
Core	`1..3` → mesh to other cores, `10+` → distribution downlinks

All host MACs on one access switch share the first 5 bytes, so the fabric matches a single **/40 masked flow** per destination access (`00:00:00:pod:acc:00 / ff:ff:ff:ff:ff:00`) — efficient forwarding.

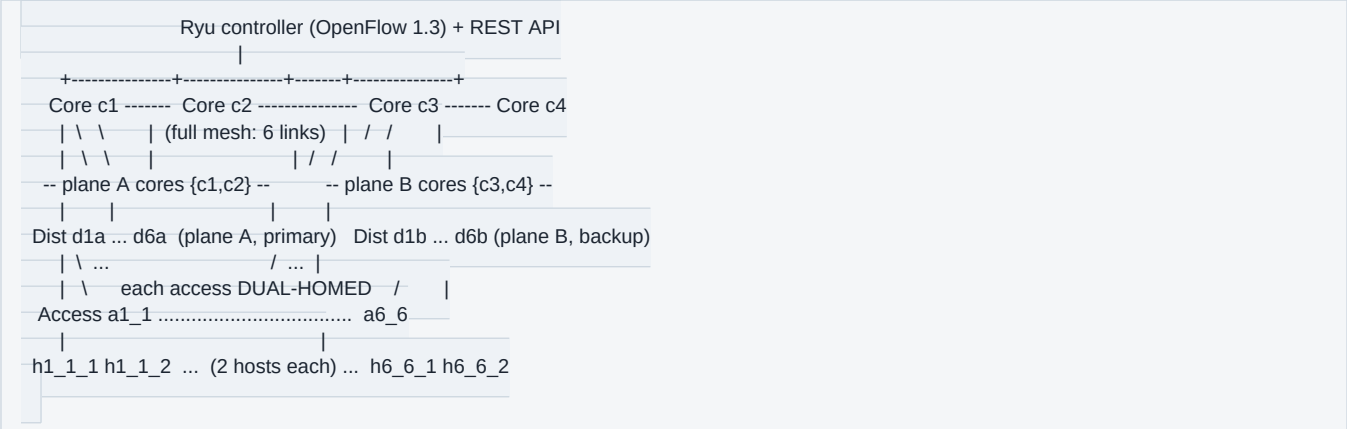
---

3. Architecture & topology design

3.1 Why a "hybrid campus" topology

Four candidates were evaluated: traditional 3-tier, spine-leaf, fat-tree, and pure Clos. The chosen design is a **hybrid campus**: a 3-tier Core/Distribution/Access hierarchy (realistic, viva-friendly, and giving *named* failure layers) hardened with data-centre-grade redundancy (a meshed core and dual-homed access). This is the only option that scores well on **all** of: campus realism, named failure classes, genuine path diversity, clean mapping to OpenFlow fast-failover groups, and manageable Mininet size.

3.2 The topology



- **Core**: 4 switches, full mesh. Split into two planes: `{c1,c2}` serve plane A, `{c3,c4}` serve plane B.
- **Distribution**: 2 per pod — `d<pod>a` (plane A) and `d<pod>b` (plane B). Each plane-A dist connects to both plane-A cores; each plane-B dist to both plane-B cores.
- **Access**: 6 per pod, each **dual-homed** to both distribution switches of its pod (plane A primary, plane B backup).
- **Hosts**: 2 per access switch, single-homed (as in a real campus).

3.3 Path diversity (the point of the design)

Between any two pods there are **two node-disjoint paths**:

```
plane A:  access → distA → core(A) → distA → access
plane B:  access → distB → core(B) → distB → access
```

Example `a1_1 → a6_6`: `a1_1-d1a-c1-d6a-a6_6` (plane A) and the fully disjoint `a1_1-d1b-c3-d6b-a6_6` (plane B). Verified with `networkx` (node-connectivity = 2).

3.4 Single points of failure (honest)

Component	SPOF?	Why
Any single link	No	dual-homing / mesh
Any single dist/core switch	No	redundant pair / mesh

Component	SPOF?	Why
An access switch	Yes — **for its own hosts only**	hosts are single-homed (normal campus)
Controller	Partial	installed flows + FF groups keep working; see §7.4

---

4. Software stack & environment

Component	Version	Runs under	Notes
Mininet	2.3.0	system `python3` (root)	emulation
Open vSwitch	3.3.9	—	OpenFlow 1.3
Ryu	4.34	**`ryu-venv` (Python 3.9)**	controller
eventlet / dnspython	0.30.2 / 1.16.0	ryu-venv	pinned — newer versions break Ryu
Node.js / npm	20.x / 10.x	—	dashboard build

> **Important:** Ryu does **not** run on Python 3.12 (eventlet dropped `wsgi.ALREADY_HANDLED`). It runs in a dedicated **Python 3.9 virtualenv** (`ryu-venv/`) with the two pinned dependencies above. Mininet keeps using the system Python 3.12. The two never mix.

---

5. Repository layout

```
OPENFLOW_PROJECT/
+-- campus_model.py       # single source of truth: switches, links, ports, hosts,
|                         #   graph helpers, spanning tree, disjoint paths
+-- topology.py           # Mininet builder + all test modes (conn/discover/failover/…)
+-- controller.py         # Ryu OF1.3 controller: forwarding, discovery, FF groups,
|                         #   rerouting, REST API, failure injection
+-- paths.py              # path computation + failure-aware forwarding planner + simulator
+-- validate_topology.py  # Phase 2 validation (wiring, disjoint paths)
+-- validate_paths.py     # Phase 5/6 validation (FF logic, offline simulation)
+-- validate_reroute.py   # Phase 7 validation (controller reroute, offline)
+-- experiments.py        # Phase 8 metrics harness -> results/*.json
+-- run_phase3.sh         # controller + baseline connectivity
+-- run_phase4.sh         # + LLDP discovery
+-- run_phase6.sh         # fast-failover demo
+-- run_phase7.sh         # controller-reroute demo
+-- run_phase8.sh         # metrics suite
+-- run_phase9.sh         # network + REST API + Mininet CLI (main "run everything")
+-- topology.json         # exported model (generated)
+-- results/              # experiment JSON (generated by run_phase8)
+-- ryu.log               # controller log (generated at run time)
+-- ryu-venv/             # Python 3.9 virtualenv for Ryu
+-- README.md             # quick reference
+-- docs/USER_MANUAL.md   # this document
+-- dashboard/            # React + TypeScript NOC dashboard (Vite)
    +-- src/…
```

Every `run_phase*.sh` takes `smoke` (default, 12 switches) or `full` (52 switches), e.g. `sudo ./run_phase9.sh full`.

---

6. How the system works

6.1 Single source of truth

`campus_model.py` builds the whole topology deterministically from four parameters (`pods`, `access_per_pod`, `hosts_per_access`, `cores`). The **same** model is imported by the Mininet builder, the controller, the validators, and the experiment harness — so the data plane and control plane can never disagree.

6.2 Controller bring-up (per switch)

When a switch connects, the controller installs:

1. **table-miss** flow → send unknown packets to the controller.
2. **IPv6 drop** — this is an IPv4 project; dropping IPv6 removes noisy host boot-time multicast.
3. **Broadcast tree flows** — one flow per ingress port, forwarding broadcast only along a pre-computed **spanning tree** (see §6.4).
4. requests **port descriptions** (to cache hardware addresses for failure injection later).

6.3 Unicast forwarding (protected)

The first packet of a flow hits the table-miss and reaches the controller. The controller then installs **destination-rooted protected forwarding** toward the destination's access switch:

- Every fabric switch gets a **/40 masked flow** (one per destination access) whose action is an **OpenFlow fast-failover group** (primary port + backup port), or a plain output where no local backup exists.
- The destination access switch gets **exact-MAC** flows to deliver to its hosts.

Because forwarding is *destination-rooted* and follows shortest paths, it is loop-free by construction.

6.4 Loop-free broadcast (no spanning-tree protocol)

The topology is intentionally redundant, so naive flooding would storm. Instead the controller computes **one spanning tree** over the switch graph and installs per-ingress broadcast flows that output only to that switch's **tree ports + host ports, minus the ingress port**. Broadcast therefore stays on the acyclic tree — no loops, no storm, and no chatty STP. (OpenFlow 1.3 has no per-port `NO_FLOOD` flag, so this explicit approach is used instead of `OFPP_FLOOD`.)

6.5 MAC / host learning

Access switches copy host-originated broadcasts (ARP) to the controller, which records `host MAC → (access switch, port)` and emits `HOST_LEARNED`. **Learning only trusts host-port ingress** — a host MAC seen on an inter-switch (transit) port is ignored, so the learned table can never place a host at the wrong switch. (Forwarding also has an authoritative fallback: host locations are known from the model.)

6.6 Topology discovery & reconciliation

Run with `--observe-links`, the controller uses **LLDP** (Ryu's topology module) to *discover* switches and links from the live network, independently of the model. It continuously **reconciles** discovered links against the expected model links and reports `RECONCILE {expected, observed, matched, missing, extra}`. At full scale this converges to **204/204** directed links, 0 missing, 0 extra — proving the running network matches the design. LLDP `EventLinkDelete` is also the real, event-driven **link-down signal** that drives rerouting.

6.7 Structured events

Every significant action emits a JSON `EVENT` line (also kept in an in-memory ring buffer served by the API): `SWITCH_CONNECTED`, `LINK_DISCOVERED`, `HOST_LEARNED`, `PROTECTED_DEST_INSTALLED`, `PRIMARY_PATH`, `BACKUP_PATH`, `LINK_DOWN`, `FAILOVER_TRIGGERED`, `FLOW_REINSTALLED`, `LINK_RECOVERED`, `RECONCILE`, `API_LINK_DOWN/UP`, `SWITCH_DISCONNECTED/RECOVERED`.

---

7. Resilience: the two failover mechanisms

7.1 Local fast-failover (OpenFlow FF groups)

Each fabric switch that has an interchangeable next hop is programmed with an `OFPGT_FF` group:

```
group (type=ff):
bucket 0:  watch_port = primary port  → output primary   (used while port is live)
bucket 1:  watch_port = backup port   → output backup    (used when primary is down)
```

If the primary port loses carrier, the **switch itself** instantly forwards out the backup bucket — **no controller involvement**. This protects:

- **access** → **dist uplink** (backup = the other plane's dist),
- **dist** → **core uplink** (backup = the second core of the plane),
- **core** → **destination-dist** (backup = via the other core through the mesh).

Because both plane cores are interchangeable and both pod dists reach the access, these backups always still reach the destination.

7.2 Controller rerouting

Some failures have **no local backup**: the final **dist** → **access** hop (a single link), and **whole-switch** failures. Here the controller:

1. detects the failure via `LINK_DOWN` (LLDP) or `SWITCH_DISCONNECTED`,
2. adds it to a live down-set,
3. recomputes each affected destination's plan over the **live graph** (failed edges/nodes removed) using a loop-safe shortest-path sink tree,
4. reinstalls only the switches whose forwarding changed (modifying FF groups / flows) — e.g. flips the whole destination from plane A to plane B.

7.3 Which mechanism handles what

Failure	Mechanism	In the loop?
access uplink link	FF group	switch only
dist→core link	FF group	switch only
core→dist link	FF group	switch only
dist→access (last hop) link	controller reroute	controller
distribution / core switch	controller reroute	controller

7.4 Controller failure (discussed, not overclaimed)

If the controller is temporarily unavailable: **already-installed flows keep forwarding**, and **FF groups keep protecting locally** (watch-port liveness is evaluated in the switch, not the controller). What stops: learning new hosts and *global* reroutes for switch-level failures. Multi-controller HA is future work; it is **not** implemented, and the

project does not claim it.

---

8. Measured results

Generated by `run_phase8.sh` into `results/*.json` (50 packets/s ping, so 20 ms resolution). Representative smoke-scale run:

Scenario	Mechanism	Loss	Outage	Detection	Reroute	Path before → after
baseline	none	0/400	0 ms	—	—	(stable)
access uplink	local FF	1/400	**20 ms**	22 ms	4 ms	plane A → plane B
dist→core	local FF	1/400	**20 ms**	21 ms	3 ms	via 2nd core
core→dist	local FF	1/400	**20 ms**	22 ms	4 ms	via other core
last-hop	controller reroute	2/400	**40 ms**	24 ms	6 ms	plane A → plane B

Full-topology connectivity: `pingall` over 72 hosts = **5112/5112 received, 0% loss**, 0 controller exceptions.

How to read it: for the FF rows, the ~20 ms outage *is* the fast-failover; the controller's detection/reroute columns show it also reacting slightly later (it only re-optimises — FF already restored traffic). For the last-hop row, the controller *is* the mechanism, so detection + reroute ≈ the 40 ms outage. Both are well under 100 ms.

---

9. Installation & first-time setup

Prerequisites (already present on the development machine): Mininet, Open vSwitch, Python 3.9 and 3.12, Node.js 20.

1. Create the Ryu virtualenv (once):

```
cd /home/nanda/OPENFLOW_PROJECT
python3.9 -m venv ryu-venv
./ryu-venv/bin/pip install --upgrade "pip<24" "setuptools<66" wheel
./ryu-venv/bin/pip install "eventlet==0.30.2" "dnspython==1.16.0" "ryu==4.34"
./ryu-venv/bin/ryu-manager --version      # -> ryu-manager 4.34
```

2. Install dashboard dependencies (once):

```
cd dashboard && npm install
```

3. Sanity-check without root:

```
python3 validate_topology.py   # 12/12 checks pass
python3 validate_paths.py      # 6/6
python3 validate_reroute.py    # 7/7
```

---

10. Operating the system — Mininet side

This is the "network engineer" view: a live emulated network you drive from a CLI.

10.1 Start the network + controller

```
sudo ./run_phase9.sh smoke      # 12 switches   (start here)
# or
sudo ./run_phase9.sh full       # 52 switches
```

The script cleans old state, starts Ryu (controller + REST API on :8080), builds the Mininet topology, and drops you into the `mininet>` prompt. The controller log streams to `ryu.log`.

10.2 Prove connectivity

```
mininet> h1_1_1 ping -c2 h6_6_1       # first packet ~1s (install+ARP), then fast
mininet> pingall                      # full reachability (0% after warm-up)
```

> The first `pingall` may show a few `X` purely from first-contact latency (first-packet install/ARP vs the 1 s ping timeout). Run it **again** → ~0% loss.

10.3 Inspect the data plane (prove it's real OpenFlow)

```
mininet> sh ovs-ofctl -O OpenFlow13 dump-flows d1a      # installed flows
mininet> sh ovs-ofctl -O OpenFlow13 dump-groups d1a     # the FF group:
     # group_id=…,type=ff,bucket=watch_port:1,actions=output:1,
     #                 bucket=watch_port:2,actions=output:2
```

10.4 Inject a failure from Mininet, watch failover

```
mininet> h1_1_1 ping h6_6_1               # (in one xterm, or use -c 100)
mininet> link d1a c1 down                 # fail the dist→core link
     # FF group swings to the 2nd core in the switch — ping barely blips
mininet> link d1a c1 up                   # restore
```

To see the controller react (detection + reroute), tail the log in another terminal:

```
grep -E 'LINK_DOWN|FAILOVER|FLOW_REINSTALLED|LINK_RECOVERED' ryu.log
```

10.5 Automated scenario runs (no manual typing)

Each produces a self-contained result:

```
sudo ./run_phase3.sh full conn      # connectivity gate
sudo ./run_phase4.sh full           # LLDP discovery + link-down detection
sudo ./run_phase6.sh full           # local fast-failover (measured)
sudo ./run_phase7.sh full           # controller reroute (measured)
sudo ./run_phase8.sh full           # full metrics suite -> results/*.json
```

10.6 Shut down

At the `mininet>` prompt type `exit` (the script auto-cleans OVS). If anything is left running:

```
sudo pkill -9 -f ryu-manager; sudo pkill -9 -f topology.py; sudo mn -c
```

---

11. Operating the system — dashboard side

This is the "NOC operator" view: a live web console.

11.1 Start it

```
# terminal 1 — network + controller + API:
sudo ./run_phase9.sh full
# terminal 2 — dashboard:
cd dashboard && npm run dev          # http://localhost:5173
```

The dashboard proxies `/api` to the controller on `:8080` (no CORS needed).

11.2 The tabs

Tab	What it shows
Overview	health tiles (switches connected, links up/down, hosts learned, FF groups, protected destinations, uptime), reconciliation status, recent failover activity
Topology	live layered graph (core → dist → access → hosts); green = spanning-tree/up links, red dashed = down; **click a link to fail/recover it, click a switch to isolate/restore it**
Failover	the measured Phase 8 results table (path before → after, loss, outage, detection, reroute) plus a **live failover timeline**
Switches / Links / Hosts	sortable tables with inline **fail / recover** buttons
Events	filterable live event log (the controller's structured events)

11.3 Trigger a real failure from the UI

1. Go to **Topology**.
2. Click a link (e.g. a dist→core line) — it turns **red dashed**. Behind the scenes the dashboard POSTs to `/api/failures/link`, the controller sends an OpenFlow **port-down**, and the real link fails.
3. Watch the **Events** tab: `API_LINK_DOWN` → `LINK_DOWN` → `FAILOVER_TRIGGERED` → `FLOW_REINSTALLED`.
4. Click the link again to **recover** it (`LINK_RECOVERED`, link turns green).

Everything is real — no mock data. The same failure is visible from the Mininet side (`pingall`, `dump-flows`) simultaneously.

11.4 Production build (optional)

```
cd dashboard && npm run build      # static bundle in dashboard/dist/
```

---

12. REST API reference

Base URL: `http://localhost:8080` (served by the Ryu controller). All responses are JSON.

Read (GET)

Path	Returns
`/api/controller`	status, counts, `reconcile` summary
`/api/topology`	`{controller, switches[], hosts[], links[]}` (for the graph)
`/api/switches`	per-switch: dpid, tier, pod, plane, connected, groups, status
`/api/links`	per inter-switch link: ports, kind, status, on_spanning_tree
`/api/hosts`	per host: mac, ip, access switch, learned
`/api/events?since=<seq>&limit=<n>`	event ring buffer (incremental with `since`)
`/api/metrics`	Phase 8 `results/summary.json` + live down counters

Actions (POST, body = JSON)

Path	Body	Effect
`/api/failures/link`	`{"a":"d1a","b":"c1"}`	real OpenFlow port-down on the a→b link
`/api/recovery/link`	`{"a":"d1a","b":"c1"}`	restore it
`/api/failures/switch`	`{"name":"d1a"}`	isolate a switch (down all its links)
`/api/recovery/switch`	`{"name":"d1a"}`	restore it

Examples

```
curl -s localhost:8080/api/controller | python3 -m json.tool
curl -s -XPOST localhost:8080/api/failures/link -d '{"a":"d1a","b":"c1"}'
curl -s "localhost:8080/api/events?limit=15"
curl -s -XPOST localhost:8080/api/recovery/link -d '{"a":"d1a","b":"c1"}'
```

13. Full demonstration walkthrough

A clean 10-minute story for a viva or demo, showing both sides at once.

Setup (2 terminals + a browser):

```
# T1:
sudo ./run_phase9.sh full
mininet> pingall          # run twice -> 0% loss (network healthy)
# T2:
cd dashboard && npm run dev
# browser: http://localhost:5173
```

Story:

1. **Healthy network.** Dashboard → Overview: `52/52 switches`, `102/102 links`, `204/204` reconciled, ~`72/72` hosts learned. Topology tab: full green fabric.
2. **Two hosts communicate.** In Mininet: `h1_1_1 ping h6_6_1`. Dashboard Failover tab shows the primary path `a1_1-d1a-c1-d6a-a6_6`.
3. **Local link failure (fast-failover).** Dashboard Topology → click `d1a↔c1`. It goes red. The ping barely blips. Events show `API_LINK_DOWN → FAILOVER_TRIGGERED` but **no reroute needed** — the FF group swung to `c2` locally (~20 ms, 1 packet). Show `sh ovs-ofctl -O OpenFlow13 dump-groups d1a`.
4. **Last-hop failure (controller reroute).** Click `d6a↔a6_6`. FF can't help (single link). Events show `LINK_DOWN → FAILOVER_TRIGGERED → FLOW_REINSTALLED` — the controller reroutes the whole flow to plane B (`…d6b…`, ~40 ms).
5. **Recover.** Click both links again → `LINK_RECOVERED`, fabric green again, reconcile back to `204/204`.
6. **Show the numbers.** Failover tab: FF ~20 ms/1 pkt vs controller ~40 ms/2 pkt — quantified resilience, both under 100 ms.
7. **Event history.** Events tab: the full `failure → detection → failover → rerouting → recovery` chain, timestamped.

The one-line takeaway: *local OpenFlow fast-failover recovers most link failures in ~1 packet with no controller involvement; controller rerouting covers the rest in ~40 ms — all on a redundant campus fabric with a network-wide SDN view.*

---

14. Validation & testing

The project validates itself at three levels — two of them need **no root**:

Script	Level	Checks
validate_topology.py	model	unique DPIDs, no port collisions, connected graph, hosts single-homed, ≥2 node-disjoint paths, each failure scenario keeps the pair connected excluding the failed part, networkx cross-check
validate_paths.py	forwarding logic (offline sim)	primary/backup disjoint; all pairs deliver loop-free; each local failure delivered via FF backup; last-hop correctly blackholes (documents the reroute case)
validate_reroute.py	reroute logic (offline sim)	recompute delivers under last-hop failure (via plane B), dead distribution switch, dead core switch — failed component never used
run_phase*.sh	live (Mininet + OVS + Ryu)	real connectivity, discovery, failover, reroute, metrics

Run the offline set anytime:

```
python3 validate_topology.py && python3 validate_paths.py && python3 validate_reroute.py
```

15. Troubleshooting

Symptom	Cause / fix
Dashboard says "controller unreachable"	the controller (run_phase9.sh) isn't running, or not on :8080. Start it; the dashboard proxies /api to :8080.
grep: ryu.log: No such file	you're in a subfolder (e.g. dashboard/). ryu.log is in the project root.
First pingall shows a few X	first-contact latency (install+ARP vs 1 s ping timeout). Re-run pingall → ~0%.
Laptop froze on full (historical)	fixed — the startup broadcast storm was closed by the explicit spanning-tree flooding. Always smoke first if unsure.
ryu-manager: command not found	use ./ryu-venv/bin/ryu-manager (Ryu lives in the Python 3.9 venv, not system Python).
sudo prompts for a password	expected — Mininet needs root; type your password in the terminal.
Something stuck after a crash	sudo pkill -9 -f ryu-manager; sudo pkill -9 -f topology.py; sudo mn -c
hosts_learned < 72	benign — a host is only "learned" after it transmits; connectivity is unaffected (0% loss).

Check for real controller errors:

```
grep -c Traceback ryu.log        # expect 0
grep -E 'NO_PATH|UNKNOWN_DEST' ryu.log   # expect none
```

16. Known limitations

- **Access switch is a SPOF for its own hosts** — hosts are single-homed (standard campus). Dual-NIC hosts are out of scope.
- **Last-hop dist→access failures need the controller** (no local FF backup at a single link) — this is by design and clearly separated in the metrics.
- **Single controller** — no multi-controller HA; discussed but not implemented.
- **IPv4 only** — IPv6 is dropped to keep the data plane clean.
- **Flows persist** until a topology change; there is no idle-timeout garbage collection (fine for a demo/research system).
- `cores` is fixed at 4 and distribution at 2/pod (the two-plane invariant); the scalable knobs are `pods`, `access_per_pod`, `hosts_per_access`.

---

17. Glossary

Term	Meaning
SDN	Software-Defined Networking — a central controller programs the switches.
OpenFlow 1.3	the protocol between the Ryu controller and OVS switches.
DPID	Datapath ID — a switch's unique OpenFlow identifier.
FF group	OpenFlow fast-failover group — picks a live output bucket in the switch.
watch_port	the port an FF bucket monitors; if it goes down, the next bucket is used.
plane A / plane B	the two independent, disjoint forwarding paths (primary / backup).
pod	a "building" — one group of distribution + access switches.
LLDP	Link Layer Discovery Protocol — how the controller discovers links.
reconcile	comparing LLDP-discovered links against the intended model.
NOC	Network Operations Centre — the monitoring dashboard.
outage	the largest run of consecutive lost packets during a failure.

End of document. Companion quick reference: `README.md`. Source of truth for all numbers: `campus_model.py` and `results/summary.json`.